

BSTBPC Backend — API Reference

Bihar State Text Book Publishing Corporation · every endpoint of `BiharTextBookBackend` , what it does, what it expects, and where the frontend uses it.

Base URL (local): `http://localhost:3000` Endpoints: 52 Public: 17 Admin: 33 Developer: 2 Generated: 11 September 2026

Access levels

Public	No login needed. Used by the public website.
Admin	Needs the login cookie from <code>POST /api/auth/login</code> (the frontend sends it automatically with <code>withCredentials</code>). An <code>Authorization: Bearer <token></code> header also works.
Developer	Needs the header <code>x-dev-key: <DEV_API_KEY></code> from the backend <code>.env</code> . Not available to admins.

Request & response format

Requests and responses are JSON unless a file is uploaded (`multipart/form-data`). Successful responses contain `success: true` and the result in `data` (login and profile use `user`). List endpoints also return `pagination` or `meta` . Fields in **bold** in the tables are required.

Error codes

400	Validation failed — the response <code>message</code> names the first invalid field; <code>errors</code> lists all of them.
401	Not logged in, login expired, or wrong email / password.
403	Developer key missing or wrong (signup, reset-password).
404	Record, file or setting not found.
409	Conflict — email already registered, duplicate chapter number, or file still in use.
413	Uploaded file is larger than the allowed size.
429	Too many login attempts for this email — wait 15 minutes.
500	Unexpected server error.

Upload limits

Images (JPG/PNG/WEBP) 2 MB · Documents (PDF/DOC/DOCX) 20 MB · Videos (MP4) 50 MB · Notice PDFs sent directly 10 MB.

Endpoints by module

MODULE	ENDPOINTS	BASE PATH
Authentication	6	<code>/api/auth</code>
Books & Chapters	10	<code>/api/books</code> , <code>/api/chapters</code>
Notices & Tenders	5	<code>/api/notices</code>
Directory (people lists)	5	<code>/api/getDirectory</code> , <code>/api/getDirectoryRow</code> , <code>/api/admin/createDirectoryRow</code> , <code>/api/admin/updateDirectoryRow</code> , <code>/api/admin/deleteDirectoryRow</code>
Sections (gallery & content lists)	6	<code>/api/sections</code> , <code>/api/admin/sections</code>
Settings (single documents)	5	<code>/api/settings</code> , <code>/api/admin/settings</code>

MODULE	ENDPOINTS	BASE PATH
File uploads	5	/api/uploads , /uploads/...
Activity log	5	/api/admin/activities
Dashboard & Book Distribution	3	/api/admin/dashboard , /api/admin/distribution
Other	2	/ , /api-docs

Authentication 6 endpoints

Login uses an HttpOnly cookie named `token` (valid 7 days). Admins can only log in; creating accounts and resetting passwords is reserved for developers.

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
1	POST	/api/auth/signup	Developer	Creates a new admin account. The password is stored as a bcrypt hash. Returns 409 if the email already exists.	Body: fullName , email , password , confirmPassword , phone . Password: 8+ chars with upper, lower, number and special character.	Developers only (Swagger / curl)
2	POST	/api/auth/reset-password	Developer	Sets a new password for an existing admin. Returns 404 if the email is not registered.	Body: email , newPassword (same rules as signup).	Developers only (Swagger / curl)
3	POST	/api/auth/login	Public	Checks email and password, sets the login cookie and returns the admin. Wrong credentials → 401. Limited to 5 attempts per email every 15 minutes (429).	Body: email , password .	Login page
4	POST	/api/auth/logout	Public	Clears the login cookie, ending the session.	—	Admin navbar & sidebar “Log out”
5	GET	/api/auth/me	Admin	Returns the logged-in admin (id, email, fullName) from the login token. 401 when not logged in.	—	Admin portal session check
6	GET	/api/auth/profile	Admin	Returns the full admin profile from the database: name, email, phone, avatar, status, created date.	—	Admin → Settings → Profile

Books & Chapters 10 endpoints

Textbooks for classes 1–12. Each book can have chapters with a PDF.

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
7	GET	/api/books	Public	Lists books with their chapter count, sorted by sort order, with pagination details.	Query: classId , subject , status (Published / Draft), page (default 1), limit (default 10).	Admin → Books Management
8	GET	/api/books/class/:classId	Public	Lists every book of one class (1–12).	Path: classId (1–12).	Public Books page, Book Reader, Flipbook
9	GET	/api/books/:id	Public	Returns one book including all its chapters in order.	Path: id .	Admin book edit form, Book Reader, Flipbook
10	POST	/api/books	Admin	Creates a book.	Body: title , classId (1–12), subject , board , coverImageUrl , description , status (Published / Draft), sortOrder .	Admin → Books → Add
11	PUT	/api/books/:id	Admin	Updates a book (send only the fields to change).	Body: same fields as create, all optional.	Admin → Books → Edit
12	DELETE	/api/books/:id	Admin	Deletes a book and all of its chapters.	Path: id .	Admin → Books → Delete
13	GET	/api/books/:id/chapters	Public	Lists the chapters of a book.	Path: id (book).	—

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
14	POST	/api/books/:id/chapters	Admin	Adds a chapter to a book. The chapter number must be unique within the book (409 otherwise).	Body: chapterNumber , title , hindiTitle, pdfUrl, sortOrder.	Admin book form → chapters
15	PUT	/api/chapters/:id	Admin	Updates a chapter.	Body: chapterNumber, title, hindiTitle, pdfUrl, sortOrder (all optional).	Admin book form → chapters
16	DELETE	/api/chapters/:id	Admin	Deletes a chapter.	Path: id (chapter).	Admin book form → chapters

Notices & Tenders 5 endpoints

One table holds both, separated by `type` = Notice or Tender. Lists are sorted pinned first, then newest publish date.

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
17	GET	/api/notices	Public	Lists notices and/or tenders with pagination details.	Query: <code>type</code> (Notice / Tender), <code>category</code> , <code>isPinned</code> (true / false), <code>page</code> , <code>limit</code> (default 10, max 1000).	Public Notice, Tenders & Notice Board pages; Admin → Notices / Tenders
18	GET	/api/notices/:id	Public	Returns one notice or tender.	Path: id .	—
19	POST	/api/notices	Admin	Creates a notice or tender. The PDF can be sent as a file or as a URL (not both).	Body (JSON or multipart): title , description , <code>type</code> , <code>category</code> , <code>isPinned</code> , <code>publishDate</code> (YYYY-MM-DD), <code>documentUrl</code> — or file field <code>document</code> (PDF, max 10 MB).	Admin → Notices / Tenders → Add
20	PUT	/api/notices/:id	Admin	Updates a notice or tender. If its uploaded PDF is replaced or removed, the old file is deleted.	Body: same fields as create, all optional.	Admin → Notices / Tenders → Edit
21	DELETE	/api/notices/:id	Admin	Deletes a notice or tender and its uploaded PDF.	Path: id .	Admin → Notices / Tenders → Delete

Directory (people lists) 5 endpoints

Allowed `type` values: `leader` (Leaders), `board_member` (Board of Directors), `past_md` (List of MDs), `employee` (Our Employees — employment type Regular / Contract / Outsource is stored in `tag`), `officer`. Lists are sorted by `sortOrder`.

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
22	GET	/api/getDirectory/:type	Public	Lists all people of one type.	Path: type .	Home page leaders, Board of Directors, List of MD, Our Employees; matching admin pages
23	GET	/api/getDirectoryRow/:type/:id	Public	Returns one person of that type.	Path: type , id .	—
24	POST	/api/admin/createDirectoryRow/:type	Admin	Adds a person. Only the listed fields are accepted.	Body: name , designation , department , tag , email , phone , photoUrl , tenureFrom , tenureTo , status (Active / Inactive), sortOrder .	Admin → Leaders, Board, List of MD, Our Employee
25	PATCH	/api/admin/updateDirectoryRow/:type/:id	Admin	Updates a person (send only the fields to change).	Body: same fields as create, all optional.	Same admin pages → Edit

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
26	DELETE	/api/admin/deleteDirectoryRow/:type/:id	Admin	Removes a person.	Path: type , id .	Same admin pages → Delete

Sections (gallery & content lists) 6 endpoints

Allowed `module` values: `gl-photo` (Photo Gallery), `gl-video` (Video Gallery), `gl-press` (Press Release), `tr` (Tools & Resources), `cl` (Latest Initiatives), `opmp`, `dc-reg-forms`, `csr`.

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
27	GET	/api/sections	Public	Lists items of a module with pagination details. Items with a future publish date are hidden unless <code>includeDrafts=true</code> .	Query: module , category , search , fromDate , toDate , page , pageSize (max 100), includeDrafts .	Public galleries, Press Release, home Tools & Resources grid; admin editors
28	GET	/api/sections/:id	Public	Returns one item.	Path: id .	—
29	POST	/api/admin/sections/reorder	Admin	Changes the display order of several items at once.	Body: items = [{ id , sortOrder }].	—
30	POST	/api/admin/sections	Admin	Creates an item. Files can be uploaded directly or given as URLs from the upload API.	Body (JSON or multipart): module , title , description , content , category , imageUrl , documentUrl , videoUrl , link , fileType , publishDate , sortOrder . Files: image , document , video .	Admin → Gallery, Tools & Resources, Latest Initiatives → Add
31	PATCH	/api/admin/sections/:id	Admin	Updates an item. Replaced uploaded files are deleted.	Body: same fields as create, all optional.	Same admin pages → Edit
32	DELETE	/api/admin/sections/:id	Admin	Deletes an item and its uploaded files (204).	Path: id .	Same admin pages → Delete

Settings (single documents) 5 endpoints

Key → value store. Public keys: `dc-rti` (RTI officer), `md_message` (MD Message), `csr_policy_content` (CSR policy text), `csr_policy_doc`, `printer_registry_doc` (Empanelled Printers PDF), `site_config` (site name, support email). Values can be text or JSON.

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
33	GET	/api/settings	Public	Returns several public settings as a key → value map and as a list.	Query: keys (comma-separated; non-public keys are ignored), category .	—
34	GET	/api/settings/:key	Public	Returns one public setting (JSON values are parsed). Non-public or missing keys → 404.	Path: key .	RTI, MD Message, CSR Policy, Empanelled Printers pages; admin editors
35	PUT	/api/admin/settings/:key	Admin	Creates or replaces a setting value.	Body: value (text or JSON), category .	Admin → RTI, MD Message, CSR Policy, Settings → System
36	POST	/api/admin/settings/:key/file	Admin	Uploads the PDF for a file setting and deletes the previous file.	Keys: <code>csr_policy_doc</code> , <code>printer_registry_doc</code> . Multipart field file (PDF / DOC / DOCX, max 20 MB).	—
37	DELETE	/api/admin/settings/:key/file	Admin	Clears a file setting and deletes its file (204).	Keys: <code>csr_policy_doc</code> , <code>printer_registry_doc</code> .	—

File uploads 5 endpoints

Upload first, then save the returned url (e.g. /uploads/images/...) on a book, notice, person or section. Files are served at http://localhost:3000/uploads/...

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
38	POST	/api/uploads/image	Admin	Stores an image and returns its path, URL, name and size.	Multipart field file : JPG / PNG / WEBP, max 2 MB.	Book covers, leader & MD photos, gallery images, mission icons
39	POST	/api/uploads/document	Admin	Stores a document and returns its path and URL.	Multipart field file : PDF / DOC / DOCX, max 20 MB.	Notice & tender PDFs, chapter PDFs, CSR policy PDF
40	POST	/api/uploads/video	Admin	Stores a video and returns its path and URL.	Multipart field file : MP4, max 50 MB.	Video Gallery (uploaded video)
41	DELETE	/api/uploads	Admin	Deletes an uploaded file. Refuses with 409 while a record still uses it; 404 if missing; 204 on success.	Body: path (must start with /uploads/).	—
42	GET	/uploads/...	Public	Serves uploaded files (also available under /api/uploads/...).	Path of the file.	All pages showing uploaded images, PDFs, videos

Activity log 5 endpoints

Feed of admin actions. The backend records which logged-in admin performed each action.

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
43	GET	/api/admin/activities	Admin	Returns the latest 30 activities with the admin's name and read status.	—	Navbar bell, Notifications page, dashboard Recent Activities
44	POST	/api/admin/activities	Admin	Records an activity.	Body: action (text, max 500), type (create / edit / delete / upload / system ...).	Every admin page after a save or delete
45	PATCH	/api/admin/activities/:id/read	Admin	Marks one activity as read.	Path: id.	Navbar bell, Notifications page
46	DELETE	/api/admin/activities/:id	Admin	Deletes one activity.	Path: id.	Notifications page, Recent Activities
47	DELETE	/api/admin/activities	Admin	Clears all activities.	—	Notifications page → Clear all

Dashboard & Book Distribution 3 endpoints

Numbers shown on the admin dashboard, all calculated from the database. Distribution figures are entered by admins.

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
48	GET	/api/admin/dashboard	Admin	Returns counts (books, notices, tenders, photos, videos, press releases, employees), uploads per month for the last 12 months, content split by type, and the year's distribution rows.	Query: year (default current year).	Admin → Dashboard (cards and charts)

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
49	GET	/api/admin/distribution	Admin	Returns 12 monthly rows { month, monthName, distributed, target } for a year; missing months are 0.	Query: year (default current year).	Admin → Book Distribution
50	PUT	/api/admin/distribution/:year	Admin	Saves distributed and target figures for the given months (creates or updates each month).	Path: year . Body: months = [{ month (1–12), distributed (≥ 0), target (≥ 0) }].	Admin → Book Distribution → Save

Other 2 endpoints

#	METHOD	ENDPOINT	ACCESS	WHAT IT DOES	REQUEST INPUTS	USED BY (FRONTEND)
51	GET	/	Public	Backend landing page showing server and database status.	—	—
52	GET	/api-docs	Public	Interactive Swagger documentation where endpoints can be tried out.	—	Developers